Generative AI Assignment 1: Exploring Deep Learning

Sahrish Mustafa[1] and Roll Number: 22i0977

FAST National University of Computer and Emerging Sciences
i220977@nu.edu.pk

Abstract. This report presents the implementation and exploration of deep learning models as part of Generative AI Assignment 1. The tasks involve designing and training neural network architectures for both discriminative and generative modeling. In Question 1, a Convolutional Neural Network (CNN) is developed for image classification using the CIFAR-10 dataset [1,4], including performance evaluation, feature visualization, and a hyperparameter ablation study. In Question 2, a Recurrent Neural Network (RNN) is trained on Shakespeare text [2] for next-word prediction with custom word embeddings and ablation experiments. Finally, in Question 3, three generative models—PixelCNN, Row LSTM, and Diagonal BiLSTM—are implemented and compared following the PixelRNN framework by van den Oord et al. [5]. The report documents datasets, objectives, tasks, and results. Experimental outcomes and detailed analyses are provided in subsequent sections.

Keywords: Deep Learning · Convolutional Neural Networks · Recurrent Neural Networks · Generative Models · CIFAR-10 · Shakespeare Text · PixelRNN

1 Question 1: Implementing a CNN for CIFAR-10

1.1 Objective

The objective of this task is to build a Convolutional Neural Network (CNN) for image classification using the CIFAR-10 dataset [1,4]. The focus is on CNN architecture, feature extraction, and model evaluation.

1.2 Dataset

The CIFAR-10 dataset [4] is used for this experiment, consisting of 60,000 RGB images (32x32 pixels) across 10 classes. Dataset source: Hugging Face CIFAR-10 [1].

1.3 Instructions

The task involves:

1. Dataset preparation and preprocessing.
2. Building and training a CNN model, plotting training error per epoch.
3. Evaluating model performance with confusion matrix, accuracy, precision, recall, and F1-score.
4. Visualizing feature maps from different convolution layers.
5. Conducting an ablation study by varying hyperparameters (learning rate, batch size, number of filters, and number of layers).

1.4 Data Processing

Before training the CNN model, the CIFAR-10 dataset was pre-processed using a set of transformations. These transformations were designed to both standardize the input data and augment the training set to improve generalization.

For the **training set**, the following transformations were applied:

- **Random Crop with Padding:** Images were randomly cropped to 32×32 after applying a padding of 4 pixels, introducing positional variability.
- **Random Horizontal Flip:** Flips input images horizontally with a probability of 0.5, helping the model learn orientation-invariant features.
- **Color Jitter:** Randomly alters brightness, contrast, and saturation by up to 20%, improving robustness against lighting variations.
- **Random Rotation:** Rotates images within a range of $\pm 10°$, encouraging rotational invariance.
- **Random Affine Transformations:** Applies random shear (up to 10°) and scaling (between 0.8 and 1.2), adding geometric diversity to the dataset.
- **Conversion to Tensor:** Converts PIL images into PyTorch tensors for model compatibility.
- **Normalization:** Each RGB channel is normalized using CIFAR-10 statistics (mean: $(0.4914, 0.4822, 0.4465)$, standard deviation: $(0.247, 0.243, 0.261)$), ensuring stable optimization.
- **Random Erasing:** With probability 0.5, randomly erases rectangular regions (covering 2%–10% of the image) and fills them with random values, simulating occlusion and improving robustness.

For the **test set**, only conversion to tensor and normalization were applied, as data augmentation is not required during evaluation.

Additionally, the dataset was split into three subsets:

- **Training set:** 45,000 images (after augmentation).
- **Validation set:** 5,000 images, used for hyperparameter tuning and model selection.
- **Test set:** 10,000 images, used exclusively for final evaluation.

This preprocessing strategy ensures consistent input scaling across the dataset while also enhancing the diversity of the training samples, leading to improved generalization performance.

1.5 Implementation and Results

Model Architecture The CNN was implemented in PyTorch using a modular design. The network is composed of a sequence of convolutional blocks followed by a fully connected classifier. Each convolutional layer consists of:

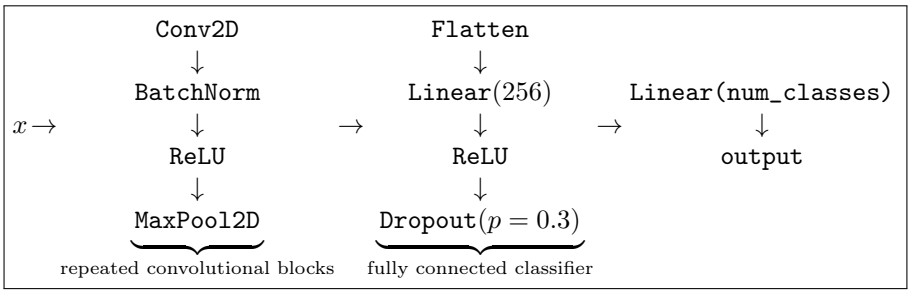
- A `Conv2d` operation with kernel size 3×3 and padding to preserve spatial dimensions.
- Batch Normalization to stabilize activations and accelerate convergence.
- ReLU activation to introduce non-linearity.

Pooling is performed after every two convolutional layers using `MaxPool2d` with pool size 2×2, progressively reducing the spatial resolution of the feature maps. This alternating pattern of stacked convolutions and pooling allows the model to learn increasingly abstract feature hierarchies while controlling computational cost.

After feature extraction, the feature maps are flattened and passed to a classifier composed of:

- A fully connected layer with 512 hidden units and ReLU activation.
- Dropout ($p = 0.5$) for regularization to reduce overfitting.
- A final dense layer mapping to the 10 CIFAR-10 output classes.

The forward pass of the CNN can be summarized as:



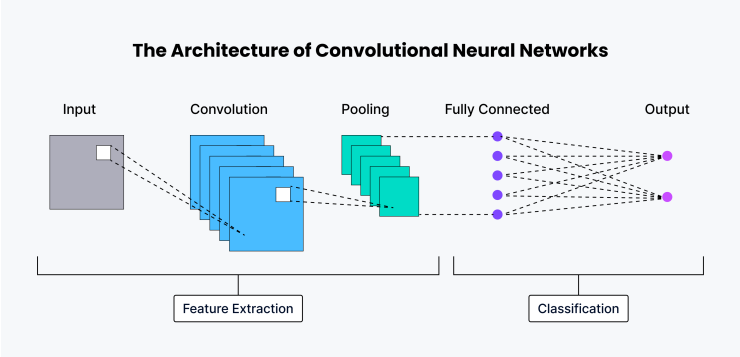


Fig. 1: Illustration of the CNN architecture used in this work, inspired by prior CNN designs [7]. The feature extraction phase consists of multiple similar blocks in sequence before the classification layer.

Training and Evaluation Procedure The training and evaluation pipeline was implemented in the function `run_experiment`, which automates the end-to-end process of training, validating, and evaluating the CNN model. The procedure consists of the following steps:

1. **Initialization:** The model is instantiated with the chosen hyperparameters and moved to the appropriate device. Cross-Entropy Loss is used as the objective function. Optimization is performed using Stochastic Gradient Descent (SGD) with momentum 0.9 and weight decay (5×10^{-4}), which together help stabilize training and act as a form of regularization. Additionally, a learning rate scheduler (`ReduceLROnPlateau`) is employed to reduce the learning rate by a factor of 0.5 if the validation loss does not improve for five consecutive epochs, further improving convergence stability.
2. **Training loop:** For each epoch:
   - The model is trained for one full pass over the training set, during which the average training loss is computed.
   - The model is evaluated on the validation set to monitor generalization performance.
   - If the validation loss reaches a new minimum, the model checkpoint is updated and the current weights are saved.
   - The scheduler is stepped using the validation loss to dynamically adjust the learning rate when progress stalls.
3. **Final evaluation:** Once training is complete, the best checkpoint (based on validation loss) is reloaded. The model is then evaluated on the held-out test set. Multiple standard metrics are reported, including:
   - Accuracy
   - Precision
   - Recall
   - F1-score
4. **Confusion matrix:** A confusion matrix is generated and saved as `confusion_matrix.png`, offering a detailed view of class-wise performance and misclassification patterns.
5. **Feature map visualization:** To provide interpretability, intermediate feature maps from selected convolutional layers are extracted and visualized. These visualizations give qualitative insights into the types of features the network is learning at different depths.

Throughout training, both training and validation losses were recorded and plotted across epochs, allowing for monitoring of convergence behavior and potential overfitting. The final model used for testing corresponds to the checkpoint with the lowest validation loss.

Feature Map Visualization To better understand what the convolutional layers of the CNN are learning, feature maps were extracted from selected layers during the forward pass. This was done by registering intermediate outputs of the network at different convolutional layers (indices 0, 2, and 4 in the `features`

module). For each chosen layer, the first eight feature maps were visualized, as shown in Figure 2.

Feature maps represent the response of convolutional filters applied to different regions of the input image. In early layers, these filters capture low-level patterns such as edges, corners, and simple color transitions. As we move deeper into the network, the feature maps become more abstract, encoding mid-level textures and eventually high-level semantic structures that are useful for classification. This hierarchical representation is a key advantage of CNNs, enabling the model to progressively build up complex features from simple primitives.

In this report, only the most informative set of visualizations is included for clarity, though feature maps were collected for multiple runs. These visualizations demonstrate how convolutional layers gradually transform the raw pixel data into structured feature representations that support the final decision-making of the classifier.

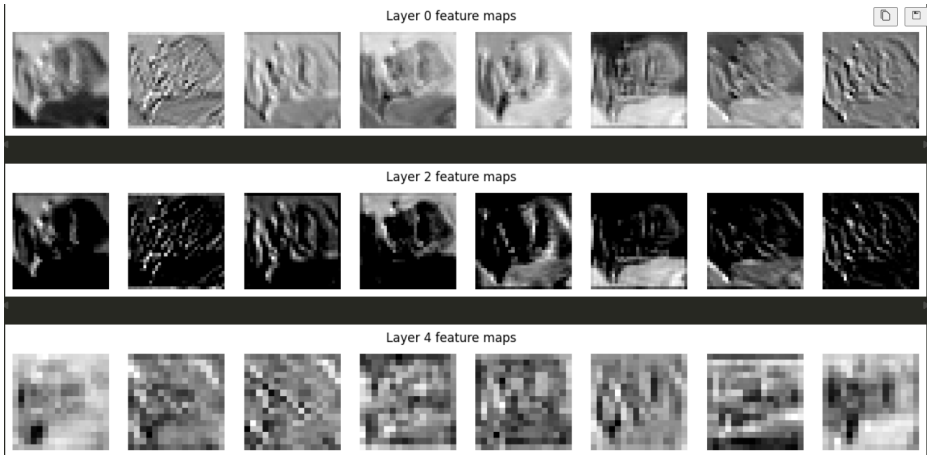


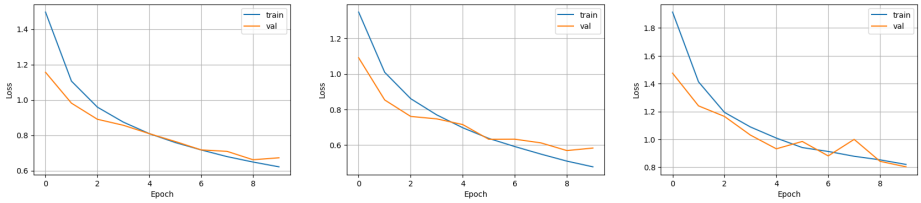
Fig. 2: Example feature maps extracted from selected convolutional layers of the CNN. Early layers capture edges and simple textures, while deeper layers highlight more abstract and task-relevant features.

1.6 Ablation Study and Experimental Results

To better understand the effect of hyperparameters on CNN performance, an ablation study was conducted. Four hyperparameters were systematically varied: learning rate, batch size, number of convolutional filters, and number of convolutional layers. For each setting, the model was trained, and results were recorded in terms of accuracy, precision, recall, and F1-score, along with qualitative evaluations (loss curves, confusion matrices, and feature maps).

Learning Rate The learning rate controls the step size in gradient descent. Models were trained with learning rates of 0.001, 0.01, and 0.1. At lower rates, convergence was stable but slower, while higher rates led to oscillations and poor

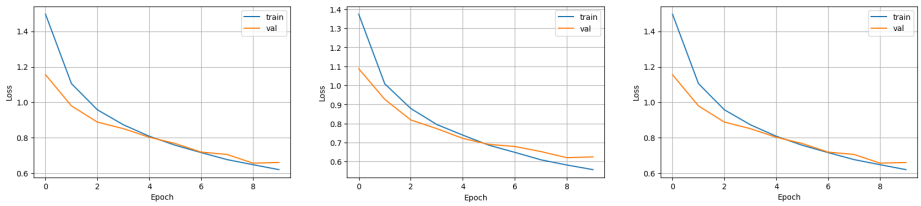
generalization. **In conclusion, learning rate of 0.1 was to be avoided due to the high loss observed when using it.**



(a) Learning rate = 0.001 (b) Learning rate = 0.01 (c) Learning rate = 0.1

Fig. 3: Training and validation loss across epochs for different learning rates.

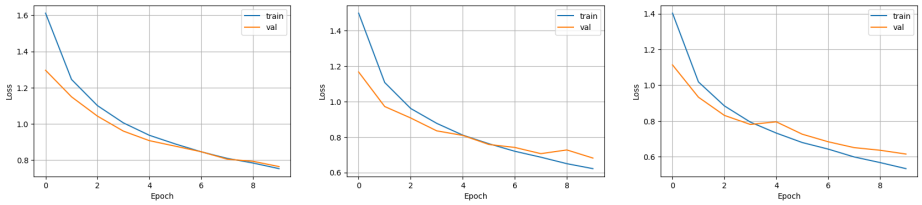
Batch Size Batch sizes of 16, 32, and 64 were tested. Smaller batches provided more stochastic updates, improving generalization but at the cost of slower training. Larger batches trained faster but sometimes converged to suboptimal minima. **This line of testing resulted in a relatively inconclusive result, but given that the default batch size used for all experiments was 16, and it did lead to the optimal result, batch size 16 was preferred in the end.**



(a) Batch size = 16 (b) Batch size = 32 (c) Batch size = 64

Fig. 4: Training and validation loss across epochs for different batch sizes.

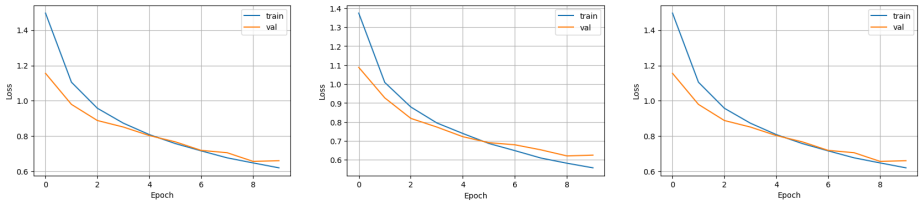
Number of Convolutional Filters The convolutional filter count was varied between 16, 32, and 64. Fewer filters led to underfitting due to limited feature extraction, whereas higher filter counts captured richer features but increased computational cost. **In general, different number of filters were still tested as the layers were increased, which provided further insight on which number of filters to use. In the end, filter count starting with 64,128.. were used.**



(a) Filter count = [16,32,64] (b) Filter count = [32,64,128] (c) Filter count = [64,128,256]

Fig. 5: Training and validation loss across epochs for different number of Convolutional Filters.

Number of Layers Experiments with 3, 5, and 7 convolutional blocks were conducted. **Deeper architectures demonstrated stronger representational capacity but were prone to overfitting without regularization. As previously mentioned, regularization techniques were used going forward, which allowed us to improve performance significantly when using more layers.**



(a) Layers=3 (b) Layers=5 (c) Layers=7

Fig. 6: Training and validation loss across epochs for different number of layers.

Quantitative Results Table 1 summarizes the quantitative metrics across all experiments. Accuracy, precision, recall, and F1-score are reported.

Table 1: Performance Metrics Comparison of CNN Models (Ablation Study)

Experiment	Accuracy	Precision	Recall	F1-score
lr_0.001	0.7673	0.7691	0.7673	0.7673
lr_0.01	0.8048	0.8087	0.8048	0.8048
lr_0.1	0.7361	0.7492	0.7361	0.7393
bs_16	0.7945	0.7979	0.7945	0.7938
bs_32	0.7868	0.7946	0.7868	0.7888
bs_64	0.7704	0.7715	0.7704	0.7698
filters_16	0.7240	0.7256	0.7240	0.7209
filters_32	0.7694	0.7702	0.7694	0.7690
filters_64	0.7773	0.7834	0.7773	0.7753
layers_3	0.7696	0.7786	0.7696	0.7716
layers_5	0.7839	0.7934	0.7839	0.7862
layers_7	0.7890	0.7902	0.7890	0.7881

Optimal Configuration Based on the ablation study, the optimal configuration was:

- Learning Rate = 0.01
- Batch Size = 16
- Number of Filters = $[64, 128, 256, 512]$
- Number of Convolutional Blocks = 8

A final model was trained using these optimal hyperparameters, achieving the best performance overall. Its results are summarized in Table 2. It was trained for an extra 30, and so a total of 50 epochs.

Table 2: Performance metrics of the CNN model trained with optimal hyperparameters.

Model	Accuracy	Precision	Recall	F1-score
Optimal CNN	0.9135	0.9134	0.9135	0.9128

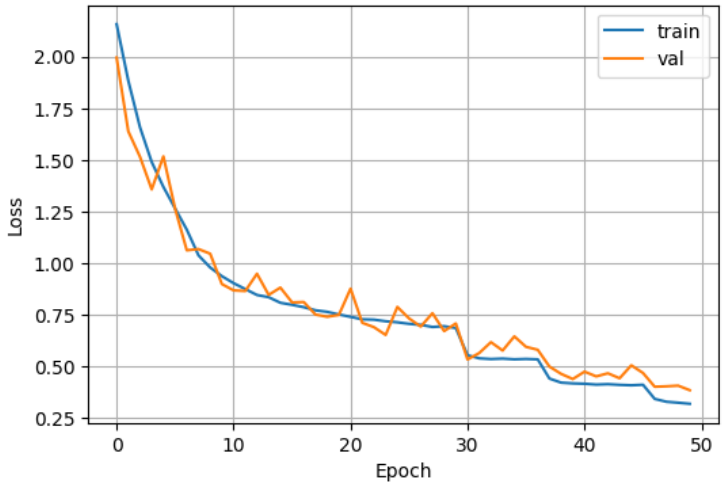


Fig. 7: Training and validation loss and accuracy across epochs for the CNN model with optimal hyperparameters.

Qualitative analysis through the loss/accuracy curves (Figure 7) and confusion matrix (Figure 8) confirmed that the optimized model learned stronger and more discriminative feature representations compared to baseline settings.

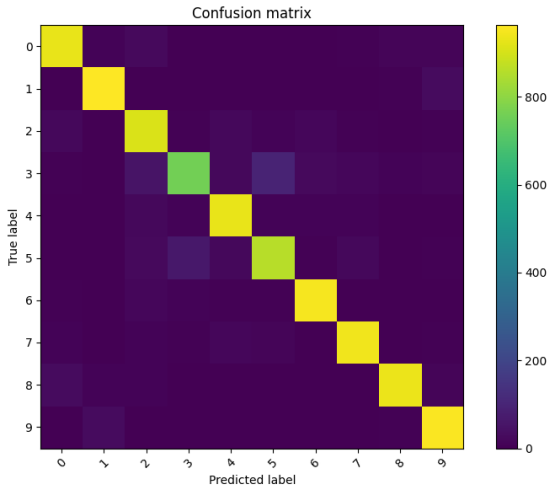


Fig. 8: Confusion matrix of the CNN model trained with optimal hyperparameters.

2 Question 2: Implementing an RNN for Next Word Prediction

2.1 Objective

The goal of this task is to train a Recurrent Neural Network (RNN) on the Shakespeare text dataset [2] for next-word prediction, with custom embeddings trained from scratch.

2.2 Dataset

The dataset used is the Tiny Shakespeare dataset from Hugging Face [2].

2.3 Tasks

The task involves:

1. Dataset loading and preprocessing.
2. Implementing an RNN model (LSTM or GRU).
3. Training the model and plotting training/validation curves.
4. Generating text predictions given a seed phrase.
5. Evaluating with metrics such as perplexity and accuracy.
6. Conducting an ablation study (e.g., hidden size, layers, dropout).

2.4 Data Preprocessing

Before training the RNN, the raw Shakespeare text needed to be cleaned and converted into a form suitable for modeling. The preprocessing pipeline consisted of the following steps:

1. **Tokenization:** A custom tokenizer (`simple_word_tokenize`) was implemented. This function splits the text into words while treating punctuation marks (e.g., ",", ".", "?") as separate tokens. This ensures that punctuation is preserved, allowing the model to learn syntactic structure.
2. **Normalization:** All tokens were lowercased to reduce vocabulary size and avoid treating "The" and "the" as different tokens.
3. **Vocabulary construction:** A vocabulary was built using `build_vocab`, which counts token frequencies. Tokens occurring below a minimum frequency threshold (`min_freq`) were excluded. Two special tokens were added:
   - `<pad>` for padding context windows.
   - `<unk>` for unknown tokens.

   This step produced two mappings: `token_to_idx` (word → index) and `idx_to_token` (index → word).
4. **Integer encoding:** The dataset text was converted into sequences of integers based on the vocabulary mapping. Out-of-vocabulary tokens were mapped to `<unk>`.
5. **Context windows:** A sliding window approach was used to construct training samples. Each input sequence consisted of a fixed `context_size` (e.g., 5) tokens, and the target label was the next token in the sequence.
6. **Train-validation split:** The dataset was split into training (90%) and validation (10%) subsets to enable model evaluation on unseen data.

This preprocessing ensured that the model received structured input in the form of numerical tensors, while preserving important linguistic features such as punctuation and word order.

2.5 Implementation and Results

Model Architecture The model was implemented as a recurrent neural network based on Long Short-Term Memory (LSTM) layers. Its architecture is composed of three main components:

- **Embedding layer:** Each word in the vocabulary is mapped to a dense vector representation of fixed dimension. This allows the model to capture semantic relationships between words rather than treating them as independent symbols.
- **Recurrent layer:** An LSTM processes sequences of embeddings. This layer is designed to capture temporal dependencies and context across multiple tokens, addressing the vanishing gradient issues common in vanilla RNNs. The model supports stacking multiple LSTM layers and applies dropout regularization between them to reduce overfitting.
- **Output layer:** The final hidden state of the LSTM (corresponding to the last word in the input context window) is passed through a fully connected linear layer that projects it onto the vocabulary space. The resulting logits represent unnormalized scores for the next word.

Training Procedure The model was trained using the cross-entropy loss function, which measures the discrepancy between the predicted distribution over the vocabulary and the true next word. Optimization was carried out with the Adam optimizer, including weight decay for regularization. Dropout was also employed within the LSTM layers to mitigate overfitting.

Training was performed over multiple epochs with mini-batches. After each epoch, the model was evaluated on a held-out validation set to monitor generalization. Early stopping was applied: if the validation loss did not improve for a fixed number of epochs (patience), training was halted to prevent overfitting. The best-performing model weights were preserved.

Evaluation metrics included:

- **Cross-entropy loss:** Used to assess training progress and generalization to unseen data.
- **Accuracy:** The proportion of correctly predicted next words.
- **Perplexity:** A standard measure for language models, defined as the exponential of the average negative log-likelihood. Lower perplexity indicates a more confident and effective model.

Hyperparameter Tuning To identify effective configurations, a range of hyperparameters were tested. Specifically, experiments were conducted with:

- **Weight decay:** 0 and 1×10^{-4}
- **Patience for early stopping:** 2 and 3 epochs
- **Top-k sampling:** 30 and 50
- **Temperature:** 0.7 and 1.0

The best-performing values from these experiments were then fixed, and further trials were carried out by varying the number of LSTM layers, the hidden size of the recurrent layer, and the dropout probability. This systematic search allowed us to balance model expressiveness with generalization, ultimately leading to the selection of an architecture that achieved the best trade-off between accuracy, perplexity, and generated text quality.

The final chosen hyperparameters for all subsequent experiments were:

- **Weight decay:** 1×10^{-4}
- **Patience:** 3 epochs
- **Top-k:** 50
- **Temperature:** 1.0

Text Generation Once trained, the model was used to generate text by predicting one word at a time, conditioned on a seed phrase. At each step, the context window was updated to include the most recently predicted word. To avoid deterministic outputs and encourage diversity, probabilistic sampling was applied:

– **Temperature scaling:** Adjusts the sharpness of the probability distribution. Lower temperatures (e.g., 0.7) make predictions more focused on high-probability words, while higher values encourage diversity.
– **Top-k sampling:** Restricts candidate predictions to the k most likely words, reducing the chance of incoherent outputs.

This combination of sampling strategies produced more fluent and stylistically Shakespearean text compared to greedy or purely random generation.

2.6 Ablation Study and Experimental Results

To better understand the effect of architectural choices on model performance, a set of ablation studies was conducted. These experiments varied one factor at a time while keeping the rest of the training setup constant, allowing us to observe how each design choice influenced loss, accuracy, and overall generalization.

Effect of Hidden Size The hidden state dimension in the LSTM controls the capacity of the model to learn complex dependencies in text. A larger hidden size increases representational power but also raises the risk of overfitting and computational cost. Two configurations were compared:

– Hidden size = 64
– Hidden size = 256

The training and validation losses for both configurations are shown in Figure 9, while the corresponding accuracy curves are shown in Figure 10. As expected, the larger hidden size achieved lower validation loss and higher accuracy, indicating improved ability to capture contextual patterns.

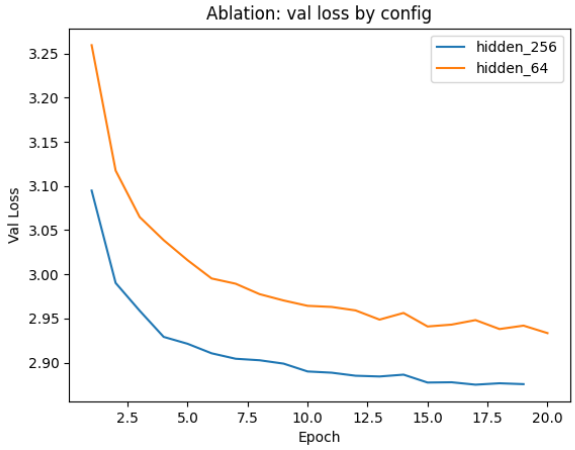


Fig. 9: Training and validation loss for different hidden sizes.

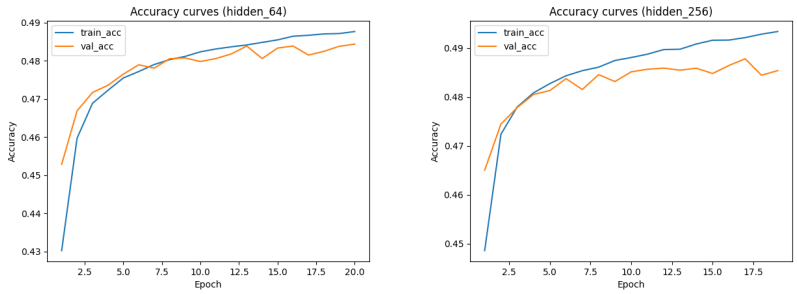


Fig. 10: Validation accuracy for hidden size 64 (left) and hidden size 256 (right).

This is also reflected in the perplexity scores: the model with hidden size 64 reached a validation perplexity of 18.79, compared to 17.74 for hidden size 256.

Effect of Dropout Dropout is a regularization technique that randomly zeroes out a fraction of activations during training, reducing overfitting. To study its effect, experiments were conducted with different dropout rates (0.1, 0.2, 0.3).

Single-layer LSTM. For the case of a single LSTM layer, the dropout parameter in PyTorch has no effect because dropout is only applied between recurrent layers when $num\_layers > 1$. As a result, all three runs with dropout values of 0.1, 0.2, and 0.3 produced nearly identical results. The differences between the runs were negligible, confirming that dropout was effectively unused in this setting.

Two-layer LSTM. When using two stacked LSTM layers, dropout is applied between the layers and therefore contributes to regularization. Experiments with dropout rates of 0.1, 0.2, and 0.3 (all with hidden size 256) showed that moderate dropout improved generalization. The plots (Figure 12) suggest that lower dropouts were the most effective balance between under- and over-regularization.

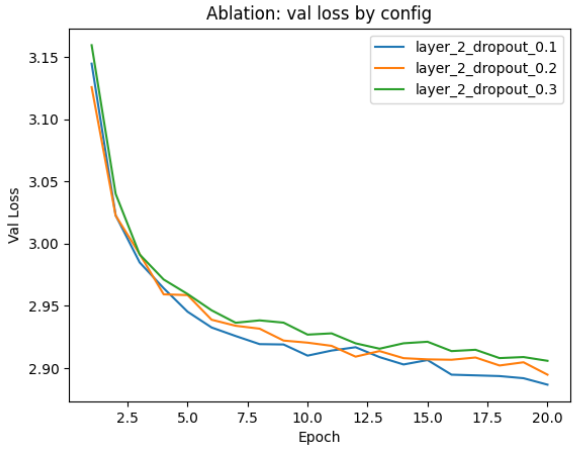


Fig. 11: Training and validation loss for two-layer LSTM with different dropout values.

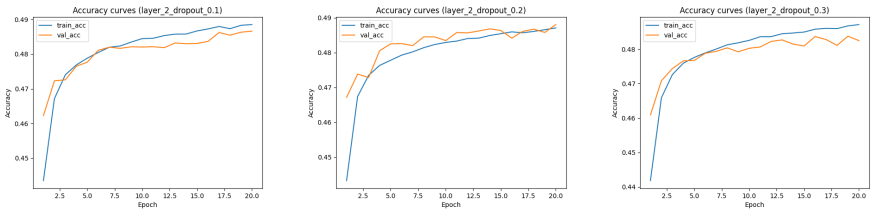


Fig. 12: Validation accuracy for two-layer LSTM with dropout 0.1 (left), 0.2 (center), and 0.3 (right).

In terms of perplexity, the two-layer models achieved values of 17.94 (0.1 dropout), 18.08 (0.2 dropout), and 18.28 (0.3 dropout), showing a slight degradation as dropout increased.

Three-layer LSTM. Finally, experiments were conducted with three stacked LSTM layers, again varying the dropout probability (0.1, 0.2, and 0.3) while keeping the hidden size fixed at 256. Compared to the two-layer setup, the deeper architecture showed a greater capacity to capture long-range dependencies, but it was also more prone to overfitting without sufficient regularization. The validation loss curves (Figure 13) demonstrate that dropout was crucial in stabilizing training, while the accuracy plots (Figure 14) highlight that a dropout rate of 0.2 provided the most consistent performance. This indicates that moderate regularization remains effective in deeper recurrent architectures, helping to strike a balance between learning capacity and generalization.

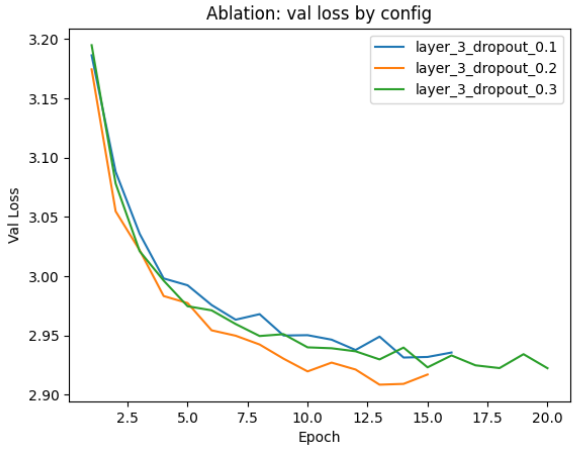


Fig. 13: Training and validation loss for three-layer LSTM with different dropout values.

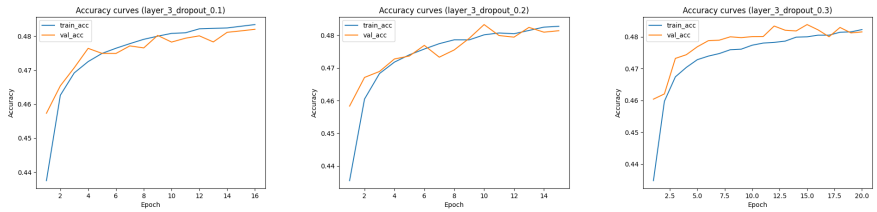


Fig. 14: Validation accuracy for three-layer LSTM with dropout 0.1 (left), 0.2 (center), and 0.3 (right).

The perplexity values for the three-layer models were very close: 17.94 (0.1 dropout), 17.99 (0.2 dropout), and 17.93 (0.3 dropout).

Perplexity Comparison To complement loss and accuracy, validation perplexity was computed for all experimental runs. Table 3 summarizes the results. Lower values indicate better language modeling performance.

Configuration	**Layers**	**Dropout**	**Perplexity**
Hidden size 64	1	0.0	18.79
Hidden size 256	1	0.0	17.74
2-layer LSTM	2	0.1	17.94
	2	0.2	18.08
	2	0.3	18.28
3-layer LSTM	3	0.1	17.94
	3	0.2	17.99
	3	0.3	17.93

Table 3: Validation perplexity for different experimental configurations.

3 Question 3: Implementing PixelCNN, Row LSTM, and Diagonal BiLSTM

3.1 Objective

The goal of this task is to implement and compare three generative models for image data based on the PixelRNN paper: PixelCNN, Row LSTM, and Diagonal BiLSTM.

3.2 Reference Paper

The main reference is van den Oord et al. (2016), *Pixel Recurrent Neural Networks* [5].

3.3 Dataset

The CIFAR-10 dataset [4] is used for training and evaluation.

3.4 Tasks

The task involves:

1. Reading and understanding the PixelRNN architectures.
2. Implementing PixelCNN, Row LSTM, and Diagonal BiLSTM.
3. Training models on CIFAR-10 with softmax over pixel values.
4. Monitoring training and validation performance with negative log-likelihood.
5. Comparing the models using evaluation metrics and generated sample quality [6,3].

3.5 Implementation and Results

Evaluation Metric: Bits per Dimension We evaluate models using **bits per dimension (bpd)**, the normalized negative log-likelihood:

$$\text{bpd} = \frac{\text{NLL(nats)}}{\ln 2 \times (\text{batch size} \times C \times H \times W)}.$$

Lower bpd values correspond to better generative modeling. A uniform predictor achieves 8.0 bpd on 8-bit images.

Masked Convolutions Masked convolutions ensure autoregressive ordering:

- **Mask A**: excludes the current pixel and all future pixels.
- **Mask B**: allows the current pixel channel while excluding future ones.

For RGB channels, masks also enforce intra-pixel ordering: $p(R)\, p(G \mid R)\, p(B \mid R, G)$.

PixelCNN Architecture The implemented PixelCNN includes:

- First layer: Mask-A convolution (7×7 kernel).
- Residual stack: 15 residual blocks, each with ReLU, Mask-B convolution (3×3), ReLU, and 1×1 conv, plus skip connections.
- Output head: ReLU $\rightarrow$ Conv (1×1) $\rightarrow$ ReLU $\rightarrow$ Conv projecting to 3×256 logits.

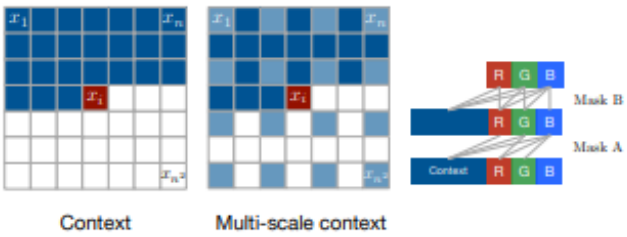


Fig. 15: PixelCNN architecture[5].

Row LSTM Architecture Row LSTM processes the image row by row:

- Input-to-state: Mask-A convolution (7×7).
- Recurrent layers: multiple `LSTMCell`s process each row, flattening width dimension to form sequences.
- Output: Concatenated row outputs projected with 1×1 conv to 3×256 logits.

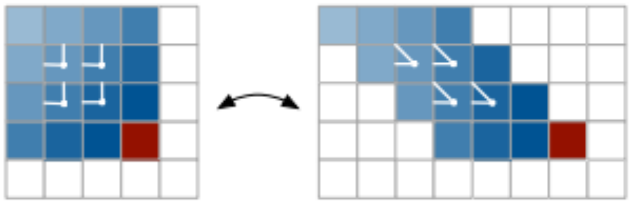


Fig. 16: Row LSTM architecture[5].

Diagonal BiLSTM Architecture Diagonal BiLSTM uses skewing and bidirectional recurrences:

- Input-to-state: Mask-A convolution (7×7).
- Extract diagonals using precomputed indices.
- For each diagonal: run forward and backward `nn.LSTM`s, then combine outputs.
- Reconstruct 2D feature map from diagonals, followed by 1×1 conv to 3×256 logits.

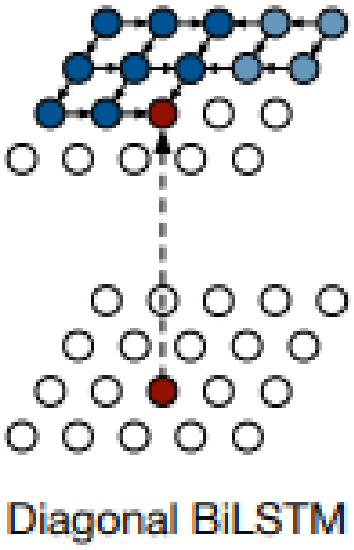
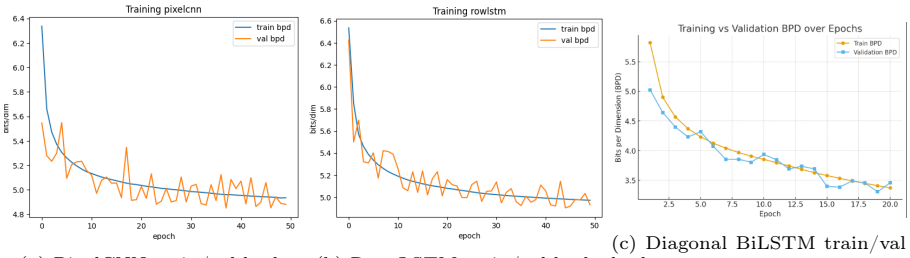


Fig. 17: Diagonal BiLSTM architecture[5].

Training Procedure

- Optimizer: RMSProp (learning rate 10^{-3}, $\alpha = 0.95$, $\epsilon = 10^{-8}$).
- Weight initialization: Xavier uniform.
- Gradient clipping: ℓ_2 norm clipped at 1.0.
- Epochs: 50, batch size: 16 (configurable).
- Loss: Cross-entropy between logits reshaped to $(B \cdot C \cdot H \cdot W, 256)$ and targets $(B \cdot C \cdot H \cdot W)$.

Results Training and validation bpd curves will be plotted for each model:



(a) PixelCNN train/val bpd. (b) Row LSTM train/val bpd. (c) Diagonal BiLSTM train/val bpd.

Fig. 18: Bits per dimension curves for each model.

Final validation bpd values are summarized in Table 4.

Model	Final Validation bpd
PixelCNN	4.8814
Row LSTM	4.9309
Diagonal BiLSTM	3.4568

Table 4: Final bits per dimension (bpd) on CIFAR-10.

Discussion

- **PixelCNN** trains faster due to convolutional parallelism, but has limited receptive field per layer.
- **Row LSTM** and **Diagonal BiLSTM** can capture longer dependencies but are slower to train and sample.

Acknowledgments

The author would like to acknowledge the support and guidance provided by FAST University faculty in completing this assignment.

Declaration of Interests

The author declares no competing interests.

References

1. Face, H.: Cifar-10 dataset. `https://huggingface.co/datasets/kriz/cifar-10` (2023)
2. Face, H.: Tiny shakespeare dataset. `https://huggingface.co/datasets/karpathy/tiny_shakespeare` (2023)
3. Heusel, M., Ramsauer, H., et al.: Gans trained by a two time-scale update rule converge to a local nash equilibrium. In: NIPS (2017) (2017)
4. Krizhevsky, A.: Learning multiple layers of features from tiny images. In: Technical report, University of Toronto (2009)
5. van den Oord, A., Kalchbrenner, N., Kavukcuoglu, K.: Pixel recurrent neural networks. arXiv preprint arXiv:1601.06759 (2016)
6. Salimans, T., Goodfellow, I., et al.: Improved techniques for training gans. In: NIPS (2016) (2016)
7. Zilliz: Convolutional neural network (cnn). `https://zilliz.com/glossary/convolutional-neural-network` (2023), accessed: 2025-09-09